ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab10: Message Passing Interface in Java

Submission Deadline: March 25th, 2026 23:59

You need at least 50 points to pass this lab assignment.

1 Introduction

The Message Passing Interface (MPI) is a **specification** for a message-passing library interface. It defines the user interface and functionality in terms of syntax and semantics, establishing the logic of the system, but it is not implementation-specific. As a result, there are multiple implementations of MPI.

Primarily, MPI addresses the message-passing parallel programming model where data is transferred from the address space of one process to another through cooperative operations on each process. MPI can run on both distributed-memory systems and shared-memory systems, encompassing a collection of processes operating either on a single computer or across multiple computers.

MPI is a widely used parallel programming paradigm due to its high portability, scalability, and good performance. It is the preferred choice for writing parallel applications on distributed memory systems, currently the most popular system deployments thanks to their interesting cost/performance ratio.

Soon after the introduction of Java, numerous implementations of MPI emerged. Among the most relevant ones, in terms of uptaken by the high performance computing (HPC) community, are **mpiJava, MPJ Express, MPJ/Ibis** and **FastMPJ**. In this lab, we will use MPJ Express to develop MPI applications in Java. MPJ Express is a reference implementation of the mpiJava 1.2 API. It is a mature and widely-used message-passing library for Java, that provides an easy-to-use and powerful interface for distributed computing. MPJ Express supports a wide range of communication protocols and can be used on a variety of platforms. MPJ Express has been around for over a decade and has a large and active user community, which means that it is well-documented and well-supported.

MPJ Express offers two configurations:

- Multicore Configuration: This configuration is used by developers who want to execute their parallel Java applications on multicore or shared-memory systems.
- Cluster Configuration: This configuration is used by developers who want to execute their parallel Java applications on distributed-memory systems including clusters and a network of computers.

We will run MPJ Express Programs with the Multicore Configuration. Instructions are in Section 4.

2 Point-to-Point Communication

Point-to-Point Communication is communication between two processes.

2.1 MPI Simple HelloWorld Program

Below are the code of the `HelloWorld.java` program:

```
public static void main(String args[]) throws Exception {
        int me, size;
        args = MPI.Init(args);
        me = MPI.COMM_WORLD.Rank();
        size = MPI.COMM_WORLD.Size();
        System.out.println(MPI.Get_processor_name() + ": Hello World from "
                           + me + " of " + size);
        MPI.Finalize();
}
```

- **Code Analysis:**

First, the MPI environment must be initialized with `MPI.Init()`:

```
public static String[] Init(java.lang.String[] argv) throws MPIException
```

During `MPI.Init()`, all of MPI's global and internal variables are constructed. For example, a communicator is formed around all of the processes that were spawned, and unique ranks are assigned to each process. After `MPI.Init()`, there are two main functions that are called: `MPI.COMM_WORLD.Rank()` and `MPI.COMM_WORLD.Size()`. These two functions are used in almost every single MPI program that you will write.

```
public int Rank() throws MPIException
public int Size() throws MPIException
```

The `MPI.COMM_WORLD.Rank()` returns the rank of a process in a communicator. Each process inside a communicator is assigned an incremental rank starting from zero. The ranks of the processes are primarily used for identification purposes when sending and receiving messages. The `MPI.COMM_WORLD.Size()` returns the size of a communicator. In our example, `MPI.COMM_WORLD.Size()` (which is constructed for us by MPI) encloses all of the processes in the job. Therefore, the call to this method returns the amount of processes that were requested for the job.

A miscellaneous and less-used function in this program is `MPI.Get_processor_name()`.

```
public static String Get_processor_name() throws MPIException
```

`MPI.Get_processor_name()` obtains the actual name of the processor on which the process is executing. The final call in this program is `MPI.Finalize()`:

```
public static void Finalize() throws MPIException
```

`MPI.Finalize()` is used to clean up the MPI environment. No more MPI calls can be made after this one.

2.2 MPI Send and Receive

Sending and receiving are the two fundamental concepts of MPI. When two proccesses communicate using MPI, one implements a `send()` operation and the other implements a `receive()` operation. Almost every single function in MPI can be implemented with basic send() and received() calls.

MPI's send() and receive() calls operate as follows: First, process A determines the message to be sent to process B. It then packs all the necessary data into a buffer designated for process B. Once the data is packed, the communication device, typically a network, routes the message to the intended destination. The destination is specified by the rank of the receiving process.

Sometimes, there are cases when A might have to send many different types of messages to B. Instead of B having to go through extra measures to differentiate all these messages, MPI allows senders and receivers to also specify message IDs with the message (known as tags). When process B only requests a message with a certain tag number, messages with different tags will be buffered by the network until B is ready for them.

2.2.1 Blocking send() operation

```
void Send(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int dest,
        int tag)
throws MPIException
```

(`buf`: data buffer
`offset`: initial offeset in sent buffer
`count`: number of items to send
`datatype`: datatype of each item in send buffer
`dest`: rank of receiving process
`tag`: message tag)

`MPI_Send()` sends the exact `count` of elements, and `MPI_Recv()`, explained later, will receive at most the `count` of elements.

For example: sending the first element of an `array` to a process with rank 1:

```
array[0] = 42;
MPI.COMM_WORLD.Send(array, 0, 1, MPI.INT, 1, 8);
```

or sending a message to a process with rank 1:

```
char[] message = "Hi there".toCharArray();
MPI.COMM_WORLD.Send(message, 0, message.length, MPI.CHAR, 1, 9);
```

The send call described above is blocking: it does not return until the message data and envelope have been safely stored away so that the sender is free to modify the send buffer. The message might be copied directly into the matching receive buffer, or it might be copied into a temporary system buffer[3].

2.2.2 Non-blocking send() operation

Use `Isend()` for non-blocking send operation:

```
Request Isend(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int dest,
        int tag)
throws MPIException
```

A non-blocking send start call initiates the send operation, but does not complete it. The send start call can return before the message was copied out of the send buffer. A separate send complete call is needed to complete the communication, i.e., to verify that the data has been copied out of the send buffer. With suitable hardware, the transfer of data out of the sender memory may proceed concurrently with computations done at the sender after the send was initiated and before it completed.

2.2.3 Blocking receive() operation

```
Status Recv(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int source,
        int tag)
throws MPIException
```

The argurments in `Recv` operation are the same as those of `send` operation except that `source` indicates the rank of the source process.

Example:

```
MPI.COMM_WORLD.Recv(data,0,1,MPI.INT,0,10);
```

The blocking receive operation returns only after the receive buffer contains the newly received message. A receive can complete before the matching send has completed. The receive buffer consists of the storage containing count consecutive elements of the type specified by datatype, starting at address `buf`. The length of the received message must be less than or equal to the length of the receive buffer. An overflow error occurs if all incoming data does not fit, without truncation, into the receive buffer. If a message that is shorter than the receive buffer arrives, then only those locations corresponding to the (shorter) message are modified.

2.2.4 Non-blocking receive() operation

Use `Irecv()` for non-blocking receive operation:

```
Status Irecv(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int source,
        int tag)
throws MPIException
```

A non-blocking receive start call initiates the receive operation, but does not complete it. The call can return before a message is stored into the receive buffer. A separate receive complete call is needed

to complete the receive operation and verify that the data has been received into the receive buffer. With suitable hardware, the transfer of data into the receiver memory may proceed concurrently with computations done after the receive was initiated and before it completed. The use of nonblocking receives may also avoid system buffering and memory-to-memory copying, as information is provided early on the location of the receive buffer.

3 Collective Communication

Unlike point-to-point communication, which is communication between two processes, **collective communication** involves participation of all processes in a communicator.

3.1 Collective communication and synchronization

Collective communication implies a synchronization point among processes. This means that all processes must reach a certain point before they can all resume execution. MPI provides `MPI.COMM_WORLD.Barrier()` dedicated to synchronize these processes. The name of the function is quite descriptive - the function forms a barrier. No processes in the communicator can pass the barrier until all of them call the function. `MPI_Barrier` can be used to synchronize a program so that portions of the parallel code can be timed accurately.

```
public void Barrier() throws MPIException
```

`Barrier()` broadcasts a message from the root process to all processes of the group. A call to Barrier blocks the caller until all processes in the group have called it.

3.2 MPI Broadcast

Broadcast is one of the standard collective communication techniques in MPI. During a broadcast session, one process sends the same data to all processes within a communicator. Broadcast is used to send user input to a parallel program or to send configuration parameters to all processes.

The communication pattern of a broadcast is shown in Figure 1. In the figure, process 0 sends the data to all other processes. Process 0 is called the root process and has the initial copy of data. All of the other processes receive the copy of data.

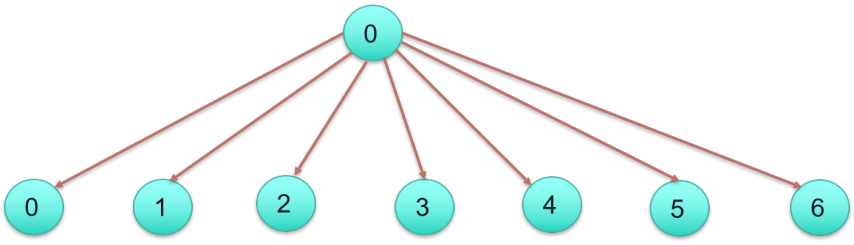

Figure 1: Broadcasting Model

In MPI, broadcasting can be done by using `MPI.COMM_WORLD.Bcast()`.

```
public void Bcast(java.lang.Object buf,
        int offset,
```

```
        int count,
        Datatype datatype,
        int root)
throws MPIException
```

`Bcast()` broadcasts a message from the process with rank root to all processes of the group. Although the root process and receiver processes do different jobs, they all call the same MPI Bcast function. When the root process calls MPI Bcast(), the `buf` variable will be sent to all other processes. When all of the receiver processes call MPI Bcast(), the `buf` variable will be filled in with the data from the root process.

3.3 MPI Scatter

MPI Scatter is a collective routine that is quite similar to MPI Bcast. MPI Scatter involves a designated root process sending data to all processes in a communicator. The primary difference between MPI Bcast and MPI Scatter is small but important. MPI Bcast sends the same piece of data to all processes while MPI Scatter sends chunks of an array to different processes. This is illustrated in Figure 2. Although the root process contains the entire array of data, MPI Scatter will copy the appropriate element into the receiving buffer of the process.

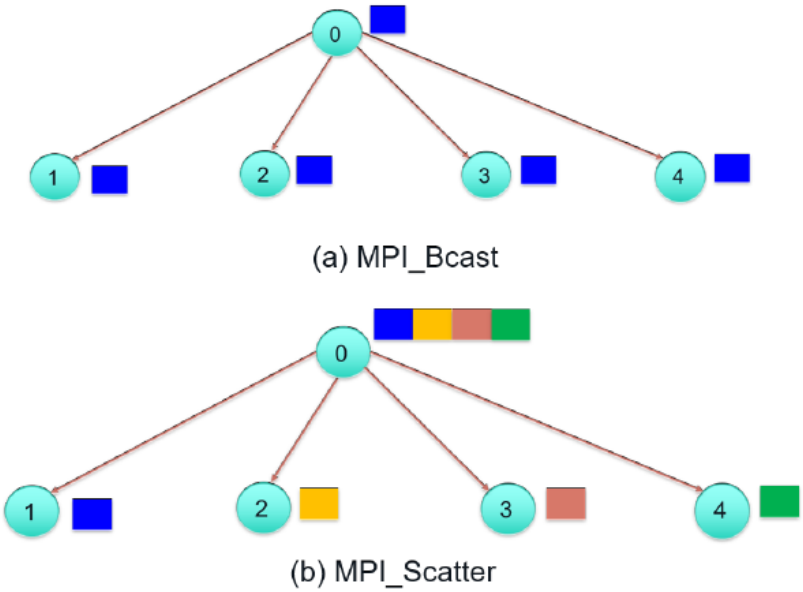

Figure 2: Broadcast and Scatter operations.

Function prototype of `MPI.COMM WORLD.Scatter`:

```
public void Scatter(java.lang.Object sendbuf,
        int sendoffset,
        int sendcount,
        Datatype sendtype,
        java.lang.Object recvbuf,
        int recvoffset,
```

```
        int recvcount,
        Datatype recvtype,
        int root)
throws MPIException
```

The first parameter, `sendbuf`, is an array of data that resides on the root process. The second, third and fourth parameters, `sendoffset, sendcount` and `sendtype`, dictate where and how many elements of a specific MPI_Datatype will be sent to each process. If `sendcount` is one and `sendtype` is `MPI_INT`, then process 1 gets the first integer of the array, process 2 gets the second integer, and so on. If `sendcount` is two, then process 1 gets the first and second integers, process 2 gets the third and fourth, and so on. In practice, `sendcount` is often equal to the number of elements in the array divided by the number of processes.

The receiving parameters of the function prototype are nearly identical in respect to the sending parameters. The `recvbuf` parameter is a buffer of data that can hold `recvcount` elements that have a datatype of `recvcount`. The last parameter, `root`, indicates the root process that is scattering the array of data. If the number of elements isn't divisible by the number of processes, then `MPI.COMM_WORLD.Scatterv` should be the one to be used:

```
public void Scatterv(java.lang.Object sendbuf,
        int sendoffset,
        int[] sendcount,
        int[] displs,
        Datatype sendtype,
        java.lang.Object recvbuf,
        int recvoffset,
        int recvcount,
        Datatype recvtype,
        int root)
throws MPIException
```

`MPI Scatterv` extends the functionality of MPI Scatter by allowing a varying count of data to be sent to each process, since `sendcount` is now an array. It also allows more flexibility as to where the data is taken from on the root, by providing the new argument, displs.

3.4 MPI Gather

`MPI Gather` is the inverse of `MPI Scatter`. Instead of spreading elements from one process to many processes, MPI Gather takes elements from many processes and gathers them to one single proces, shown in Figure 3. This routine is used to many parallel algorithms including such as parallel sorting and searching.

```
public void Gather(java.lang.Object sendbuf,
        int sendoffset,
        int sendcount,
        Datatype sendtype,
        java.lang.Object recvbuf,
        int recvoffset,
        int recvcount,
        Datatype recvtype,
        int root)
throws MPIException
```

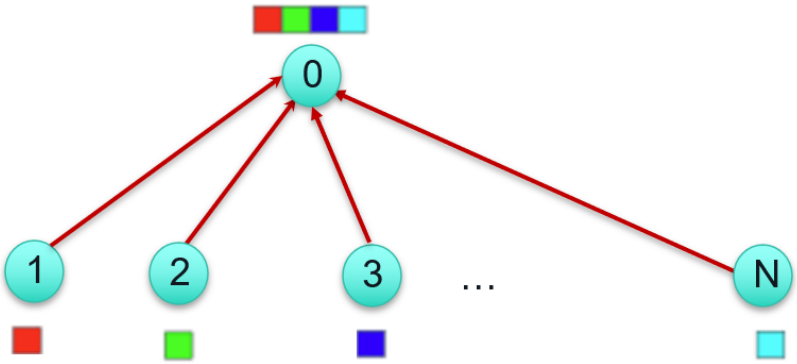

Figure 3: Gather operation

Each process sends the contents of its send buffer to the root process. In MPI Gather, only the root process needs to have a valid receive buffer. All other calling processes can pass NULL for recvbuf. The recvcount parameter is the count of elements received per process, not the total summation of counts from all processes.

Similarly to Scatter, we have another version of MPI Gather which is MPI Gatherv to be used when the numbers of elements received from each process are different.

```
public void Gatherv(java.lang.Object sendbuf,
int sendoffset,
int sendcount,
Datatype sendtype,
java.lang.Object recvbuf,
int recvoffset,
int[] recvcount,
int[] displs,
Datatype recvtype,
int root)
throws MPIException
```

4 Running MPJ Express Programs in the Multicore Configuration

There are multiple ways to run a MPJ Express program in multicore configuration. The following is a simple way to run with IntelliJ.

4.1 Setting MPJ_HOME and PATH environmental variables

There are multiple ways to run an MPJ program. If you're familiar with one, feel free to proceed with your preferred approach. Otherwise, you can follow the instructions below with IntelliJ IDEA:

1. Download MPJ Express `mpj-v0_44.zip` and unpack it.

2. Copy the folder `mpj-v0_44` to C disk in your computer and rename the folder to `mpj`. You can still keep the original `mpj-v0_44` name but then you have to modify the following steps accordingly. Now that you have MPJ Express stored in to folder C:\mpj.

If you store `mpj` in another location, you need to adjust the following steps while editing environment variables:

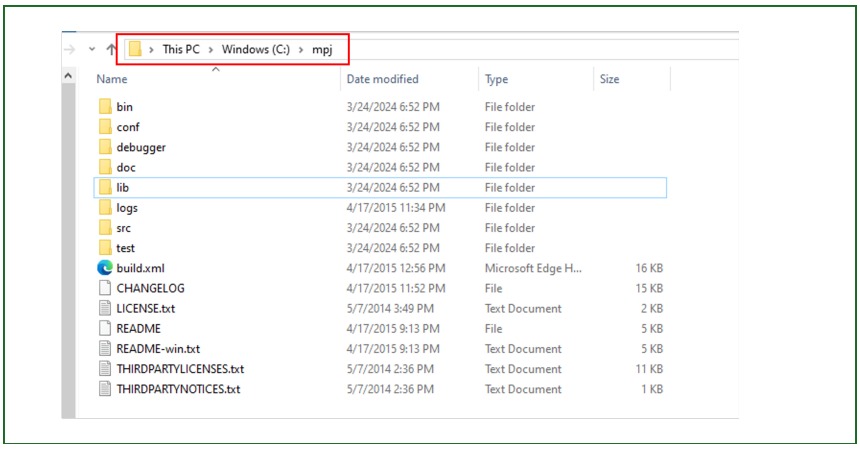

Figure 4: MPJ Express in C:\mpj.

3. Search for `System Properties` - you can do it simply by typing `sysdm.cpl` into Windows Run box if you are using Windows OS.

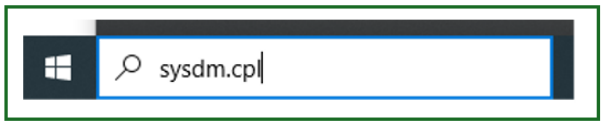

Figure 5: Search for `System Properties`,

Then, navigate to the `Advanced` tab and click on `Enviromment Variables`.

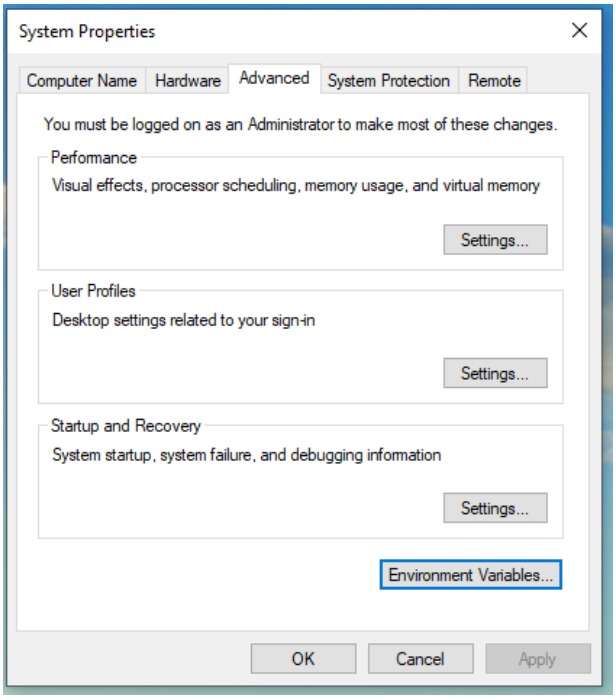

Figure 6: Then click on `Advanced` tab and `Enviromment Variables`.

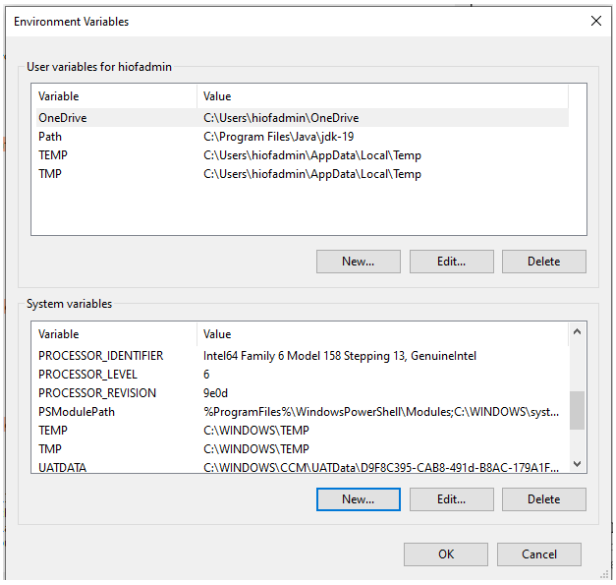

Figure 7: Add a new `Enviromment Variables`.

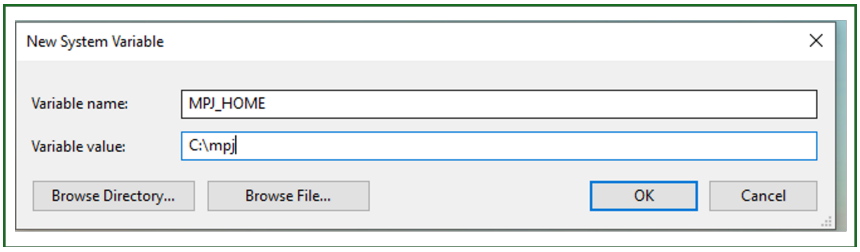

Figure 8: Add value for `MPJ_HOME` variable. If you store `mpj` in another location, modify it accordingly.

4.2 Set up Project Settings for an MPI Program in IntelliJ

1. Open a project with a MPJ program in IntelliJ.

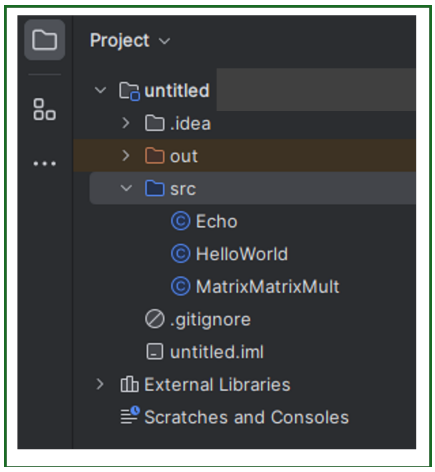

Figure 9: Open a MPJ project in IntelliJ.

2. Open `Project Structure...` with File → Project Structure or just do `Ctrl + Alt +Shift +S`

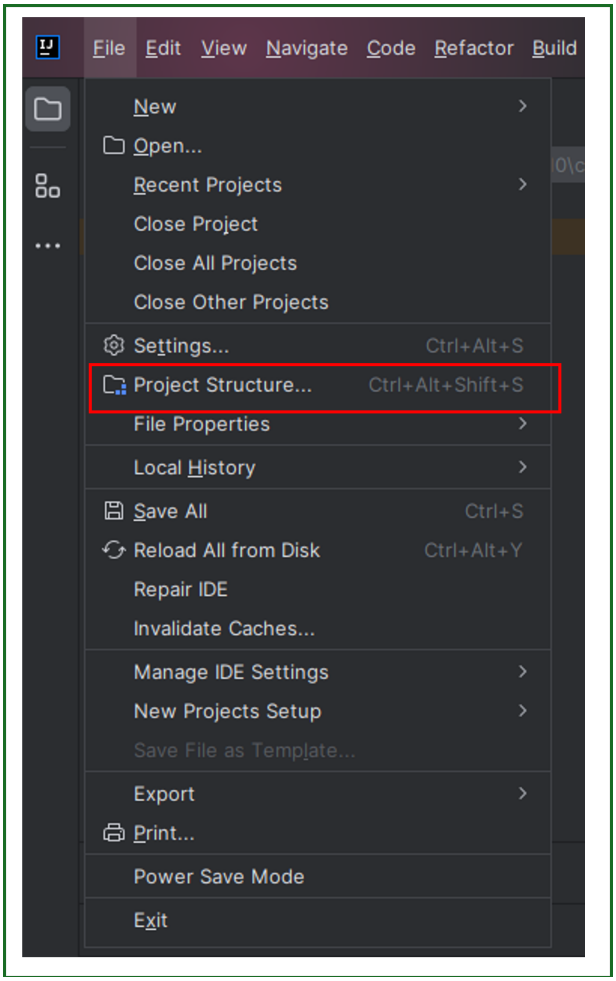

Figure 10: Select `Project Structure`.

3. Add `libraries` to the project by clicking on `+` symbol:

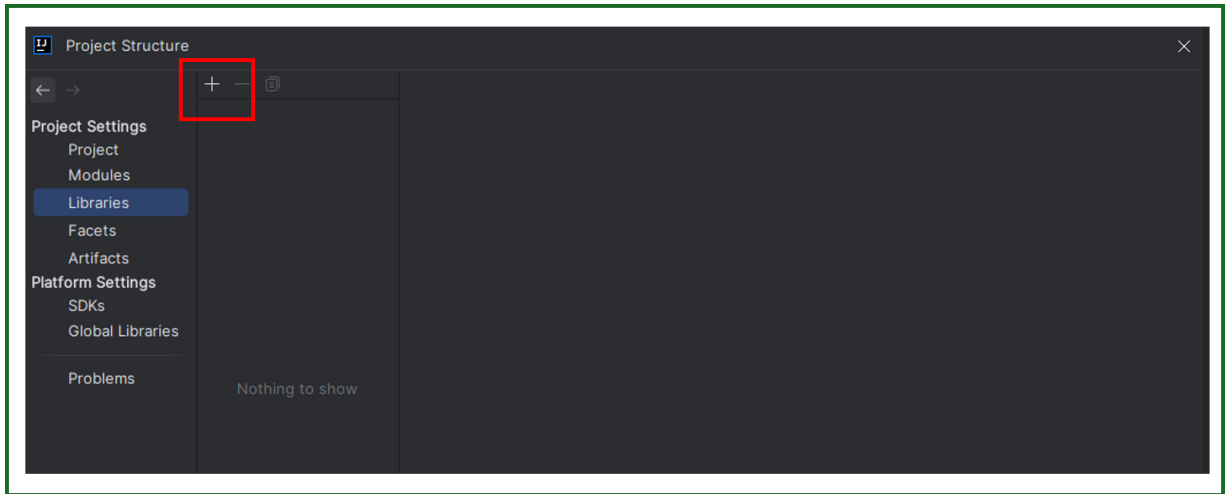

Figure 11: Adding libraries - 1.

Then:

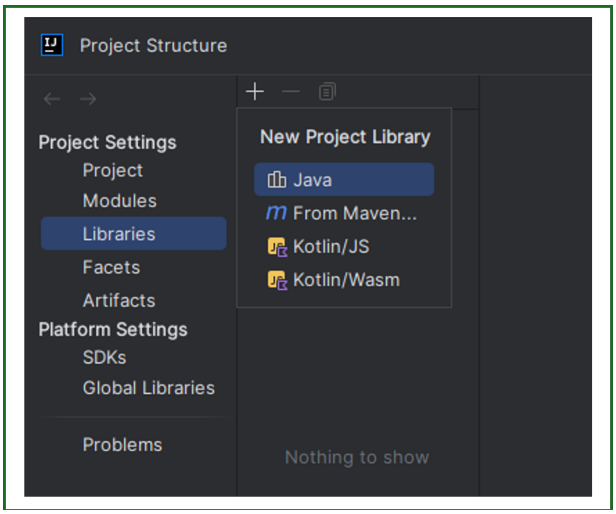

Figure 12: Adding libraries - 2.

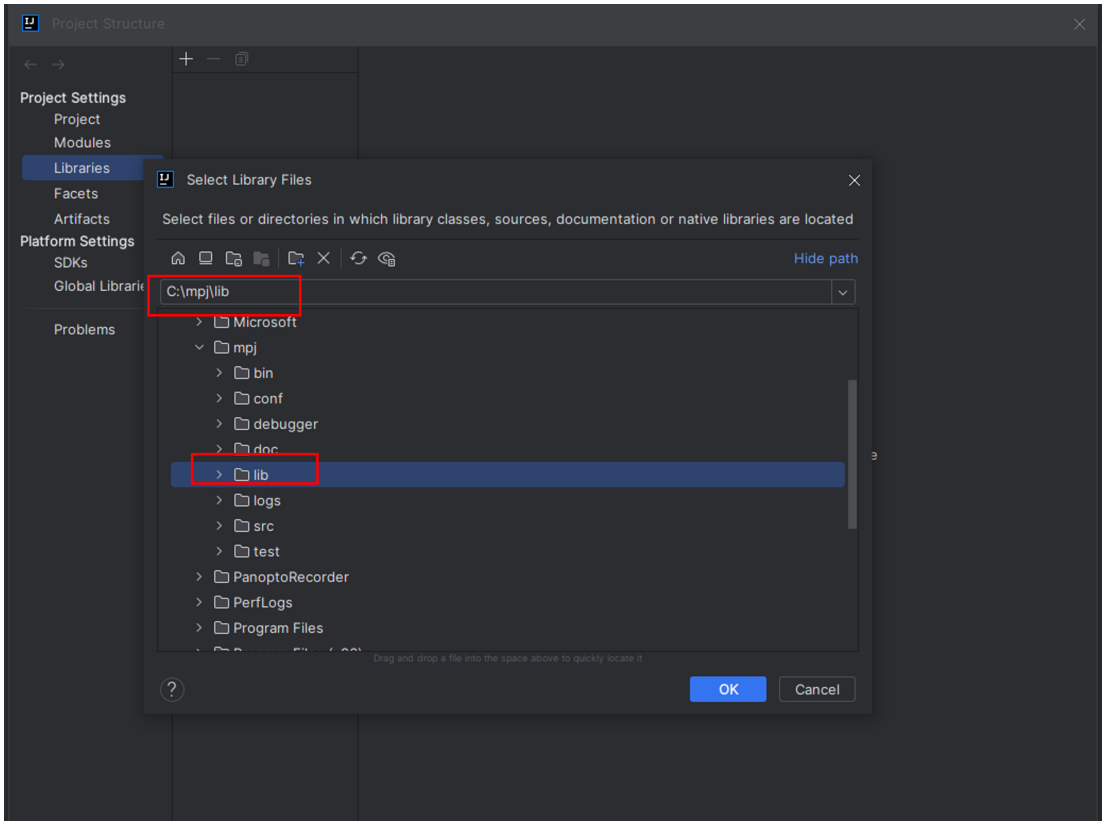

Figure 13: Adding libraries - 3.

Click on "OK" or "Apply" buttons whenever this applies. You are supposed to get the following:

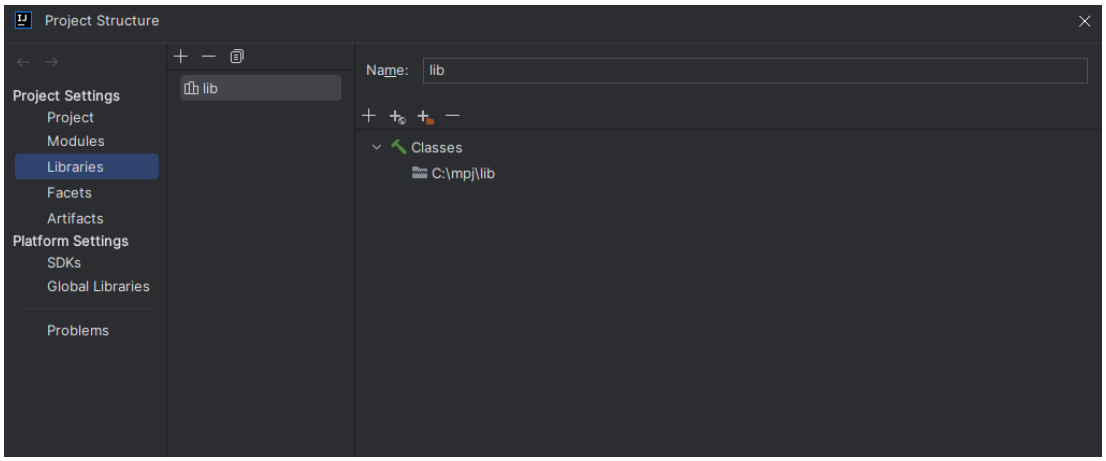

Figure 14: A correct settings.

4.3 Modify Run/debug configuration templates

Right click on the MPI program you want to run, for example `HelloWord`, in the project. Then, select `Modify Run Configuration...` as the following:

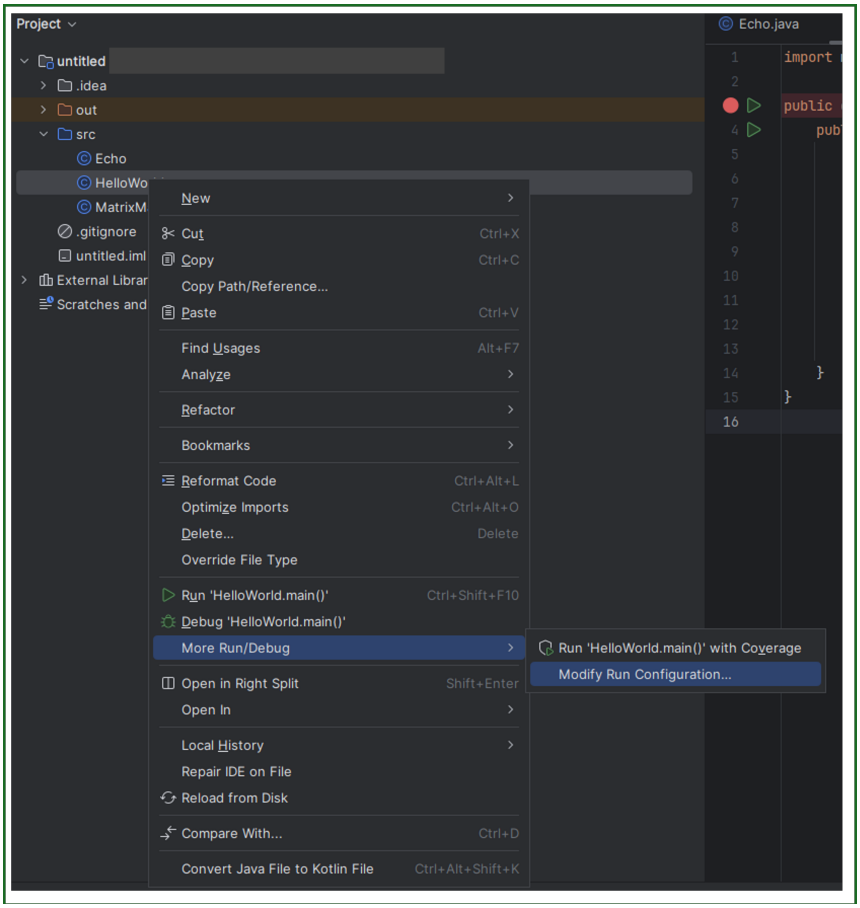

Figure 15: Select `Modify Run Configuration...`

Edit the environment variables:

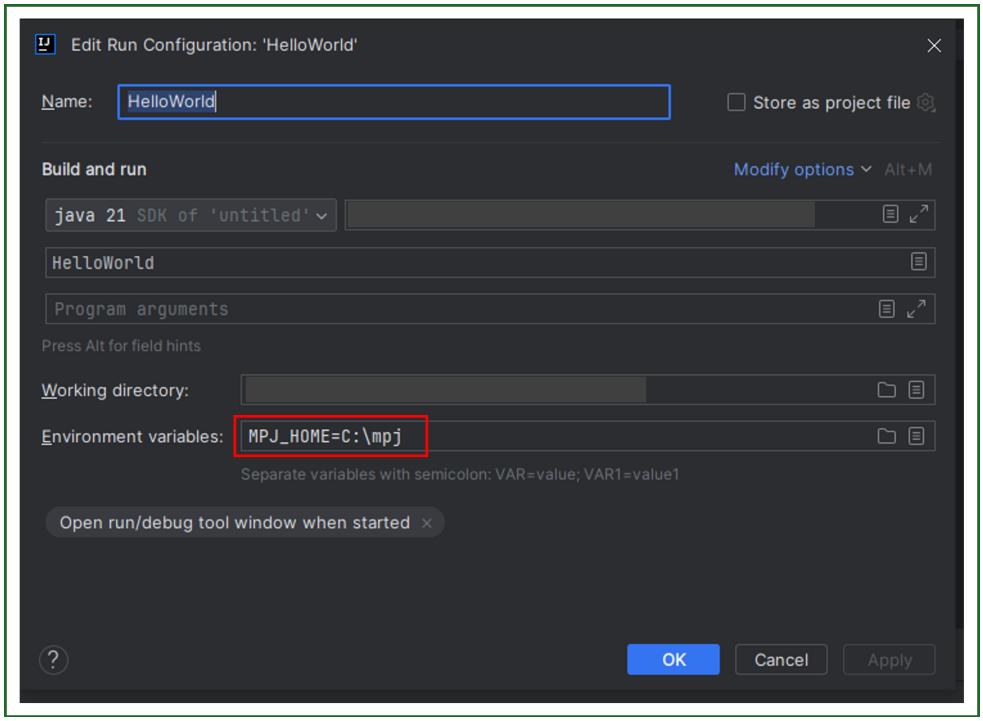

Figure 16: Add MPJ_HOME environment variable.

2. On `Modify options`, see the red box below:

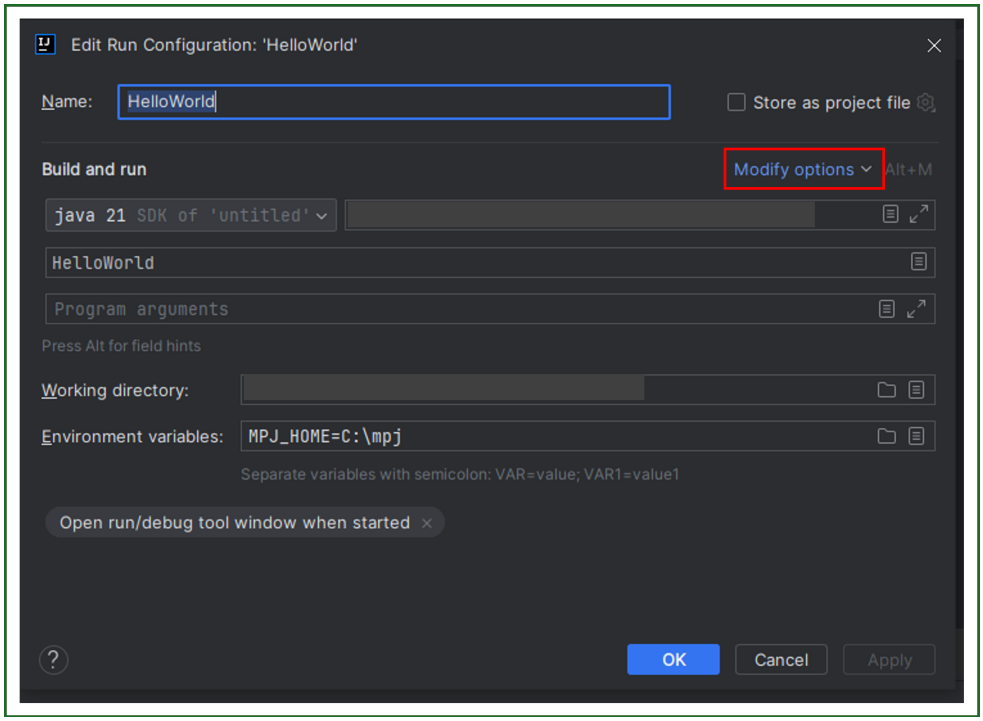

Figure 17: `Modify options`.

select `Add VM options`

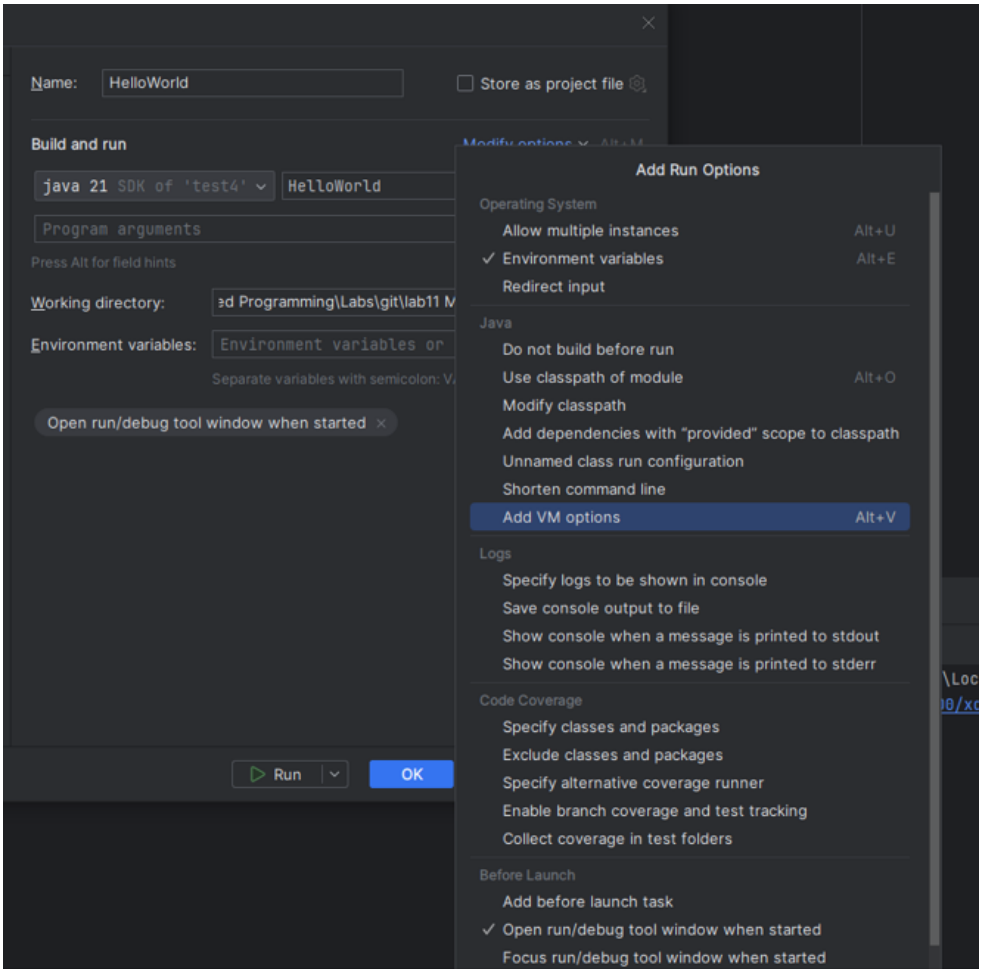

Figure 18: Select "Add VM options".

5. In the `VM Options` box, add the following command:

```
-jar $MPJ_HOME$\lib\starter.jar -np 10
```

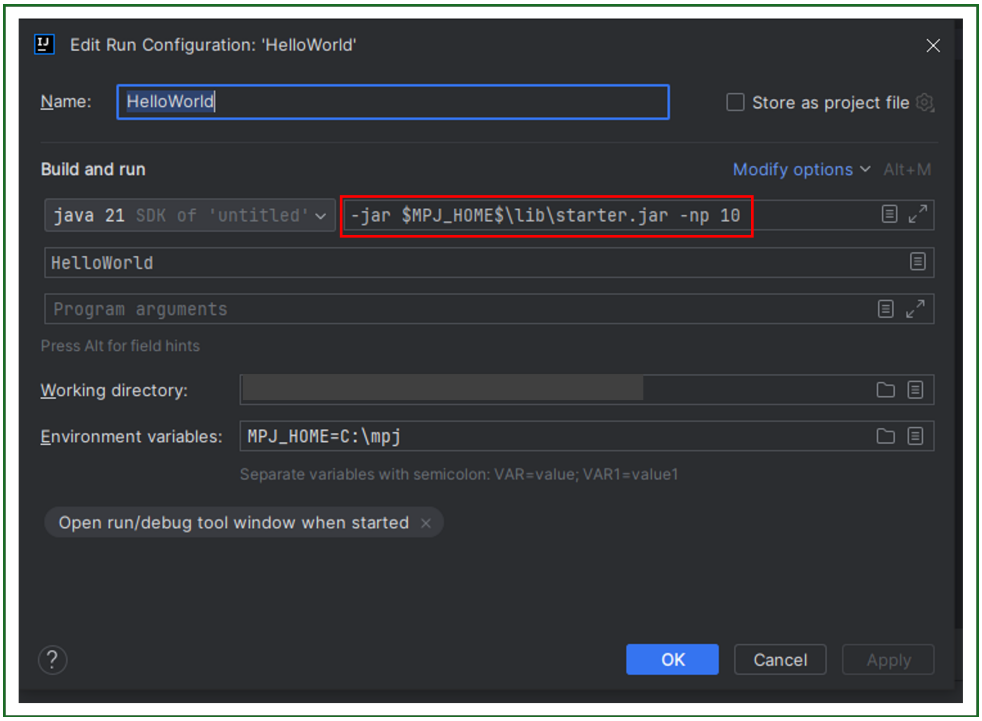

Figure 19: Create 10 processes in HeloWorld program.

In the command, `np` indicates the number of proccesses used in your program. In this case, the program runs with 5 proccesses. Output of the program with 10 procecces:

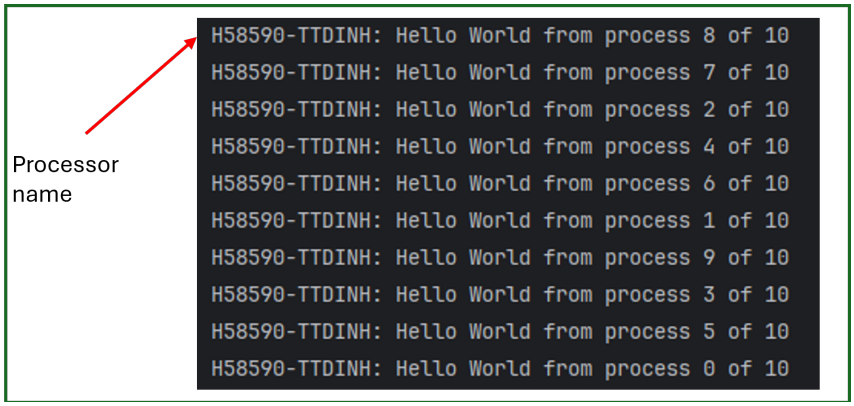

Figure 20: One possible output of `HelloWorld` java.

5 Exercises

Refer to the `untitled` project for the following exercises.

5.1 Exercise 1 (25 points)

- Run `Hello.java` with different number of processes (try few cases) and include the screenshots to your report.
- Is the program run in multicore configuration or cluster configuration?

5.2 Exercise 2 (25 points)

The `Echo.java` illustrates how two processes communicate with each other using `MPI_Send` and `MPI_Receive`.

- Run the code and include the screenshots to your report.
- In the current implementation, one process sends a random integer number to the another process. Modify the program so that 1) Process 0 send multiple message to Process 1 and when Process 1 receives the message from Process 0, it prints out the message to the screen. Run the code and include the screenshots to your report.

5.3 Exercise 3 (40 points)

The `MatrixMatrixMult.java` computes the multiplication of two square matrices ($N \times N$) using MPI.

- Analyze the code and explain how the multiplication is implemented, how tasks are divided to different processes and how the processes communicate to produce the final output.
- Run the code with $N = 1500$ and the number of processes is 1, 2,...10. Include the screenshots to your report.
- Calculate the `speedup` and discuss its values with the above setting.
- What happens when $N = 10000$? Explain your answer.

5.4 Exercise 4 (10 points)

The MPI programs in this lab are run with MPJ Express. Could the program run in other MPI standards in Java, for example `mpiJava`, `FastMPJ`, etc? Explain why or why not?

- The MPI programs in this lab are run with MPJ Express. Could the program run in other MPI standards in Java, for example `mpiJava`, `FastMPJ`, etc? Explain why or why not?
- How MPI APIs are similar and different in different MPI implementations in Java?

6 What To Submit

Complete the the exercises in this assignment. Then, put your lab10 report file (Markdown, TXT or PDF file) inside the lab10 directory of your repository. Commit your changes and remember to check the GitHub website to make sure all files have been submitted.

7 References

1.Baker, Mark; Carpenter, Bryan; Fox, Geoffrey C.; and Ko, Sung Hoon, "mpiJava: an object-oriented Java interface to MPI" (1999). Northeast Parallel Architecture Center. 7.
2. Documents about MPJ Express. https://sourceforge.net/projects/mpjexpress/files/releases/.
3. MPI 3.1 Specification. https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf.
4. http://mpjexpress.org/docs/javadocs/mpi/Comm.html#mpjdevComm